

InfoFlow Simulator: A Schema-Adaptive Telemetry Dashboard for Information Engagement Analytics

Yicheng Jiang (Team Lead, Simulation Engine & Integration)
Shuoyang Jin (Data Analysis & Predictive Modeling)
Jiaxin Wang (Dashboard Development & Deployment)
Quanquan Lu (Narrative, Theory & Communication)

2026-07-17

Table of contents

1	1. Introduction	2
1.1	Background	2
1.2	Research Question	2
1.3	Innovation Statement	3
2	2. Related Work	3
3	3. Method	4
3.1	3.1 Simulation Engine	4
3.2	3.2 Schema-Adaptive Dashboard	4
3.3	3.3 Predictive Modeling	5
3.4	3.4 Stress Testing Protocol	5
4	4. Results	6
4.1	4.1 Simulation Output	6
4.2	4.2 Dashboard Demonstration	6
4.3	4.3 Predictive Model Performance	6
5	5. Limitations and Future Work	6
5.1	5.1 Current Limitations	6

5.2	Future Work: Empirical Roadmap	7
5.2.1	Short-Term (Next Semester)	7
5.2.2	Medium-Term (CHI 2027 Target)	7
5.2.3	Long-Term Vision	7
6	6. Team Collaboration	7
7	7. Development Challenges and Solutions	8
7.1	7.1 Data Processing and Analytical Pipeline Challenges	8
7.2	7.2 Dashboard Generalization and Data Compatibility Challenges	9
7.3	7.3 System Integration and Development Workflow Challenges	10
7.4	7.4 Learning New Technical Workflows Through Collaboration	10
7.5	7.5 Lessons Learned	11
8	8. Conclusion	12
9	9. AI Usage Disclosure	12
	References	12

1 1. Introduction

1.1 Background

In the AI-driven media environment, understanding *why* audiences lose attention is as critical as understanding *what* they engage with. Traditional communication research methods—surveys, interviews, focus groups—rely on self-reported data that suffers from recall bias and social desirability effects. At the other extreme, laboratory methods such as eye-tracking and EEG provide precise behavioral measurements but are expensive and hard to scale.

This project proposes a **third path**: a lightweight, schema-adaptive telemetry framework that bridges the gap between subjective self-report and costly lab equipment. By combining synthetic behavioral simulation with an auto-configuring analytics dashboard, we demonstrate a reproducible methodology for evaluating information engagement that requires no specialized hardware and minimal technical expertise from end users. This approach is grounded in Cognitive Load Theory, which suggests that increased information complexity increases the mental effort required for processing information (Sweller 1988).

1.2 Research Question

The central research question is:

How does information complexity affect user attention retention, and can these patterns be surfaced automatically from heterogeneous behavioral logs? Previous research has investigated how information complexity influences user attention and reading behavior in digital content environments (Khot et al. 2018).

To answer this, we developed (1) a parameterized simulation engine that generates granular telemetry from virtual users navigating information structures of varying complexity, and (2) a schema-adaptive Streamlit dashboard that auto-detects column semantics, generates targeted visualizations, and trains baseline dropout-prediction models without manual configuration.

1.3 Innovation Statement

Unlike fixed-schema analytics dashboards that require a data scientist to write bespoke queries for every new dataset, **InfoFlow Simulator is a schema-adaptive telemetry engine**: it uses fuzzy column-name matching to infer data semantics, dynamically selects appropriate visualizations and statistical models, and produces a structured behavioral analysis report from *any* CSV with behavioral-log columns. This lowers the barrier from “hire a data scientist” to “upload a CSV.”

The system is validated through stress testing: five structurally diverse synthetic datasets (varying in sample size, difficulty profile, and user count) are run through the same pipeline to confirm robustness.

2 2. Related Work

Our work sits at the intersection of three literature streams:

Information Foraging & Cognitive Load. Pirolli and Card’s Information Foraging Theory (Pirolli and Card 1999) models how users navigate information spaces, optimizing for information gain per unit of cognitive cost. Sweller’s Cognitive Load Theory (Sweller 1988) posits that task complexity directly predicts performance degradation and task abandonment. Later developments of cognitive load research further distinguish different sources of mental effort involved in complex tasks (Paas et al. 2003). Our simulation engine operationalizes both frameworks: `dwel_time` proxies information-processing cost, and `dropout_flag` captures cognitive overload thresholds.

Dwell Time as Engagement Signal. Dwell time has been validated as a behavioral proxy for user interest and cognitive engagement in web search (Liu et al. 2014) and content consumption (Yi et al. 2014). Recent research has further explored dwell time as a quantitative indicator of attention and engagement in digital browsing environments (Epstein et al. 2022). Our dashboard inherits this tradition but adds automated schema detection so the metric is surfaced regardless of what the column is named in the uploaded file.

Dashboard Design & Sensemaking. Few (Few 2006) established design principles for analytical dashboards; Saraiya et al. (Saraiya et al. 2005) introduced insight-based evaluation for visualization tools. Our dashboard design also follows Shneiderman’s overview–zoom–details–on-demand principle for interactive visualization systems (Shneiderman 1996).

3 3. Method

3.1 3.1 Simulation Engine

The simulation engine (`data/program.py`) generates synthetic behavioral telemetry through an agent-based model with configurable parameters (Table 1).

Table 1: Key simulation parameters (full list in `config.yaml`).

Parameter	Meaning	Default
<code>RANDOM_SEED</code>	Reproducibility anchor	710
<code>TARGET_LOGS</code>	Total behavioral records	2000
<code>NUM_USERS</code>	Virtual user pool	130
<code>MAX_STAGE</code>	Information complexity tiers	5
<code>BASE_DWELL_TIME</code>	Baseline reading time (s)	2.5
<code>BASE_ACCURACY</code>	Baseline comprehension rate	0.85
<code>FAMILIARITY_ACC_BOOST</code>	Domain knowledge → accuracy gain	0.04
<code>DENSITY_DWELL_PENALTY</code>	Information density → slowdown	0.06

Each virtual user is assigned individual-difference parameters (speed factor, ability factor, domain familiarity) drawn from normal distributions. Users navigate 1–5 stages of increasing information complexity, with per-click outcomes (correct/incorrect) and per-stage dropout probabilities modulated by difficulty and user traits. The simulation design is also informed by Information Foraging Theory, which describes how users navigate information spaces by balancing expected information value against cognitive effort (Pirolli 2007).

All parameters are externalized to `config.yaml`, enabling stress testing without code modification.

3.2 3.2 Schema-Adaptive Dashboard

The Streamlit dashboard (`app.py`) implements a four-tab architecture:

1. **Data Overview** — column quality audit (missingness, dtypes, distributions), raw preview

2. **Behavioral Analysis** — adaptive charts selected by detected column semantics (heatmap, retention curve, density-dropout scatter, familiarity bar chart, action type pie, correlation matrix)
3. **Predictive Model** — auto-trained logistic regression on dropout prediction, with confusion matrix and coefficient-based feature importance
4. **Future Directions** — research roadmap from current prototype to CHI 2027 submission

The key technical contribution is the `infer_schema()` function, which uses a keyword-alias dictionary to map arbitrary column names (e.g., `Duration_sec`, `time_on_page`, `reading_time`) to canonical semantic roles (`dwelt_time`). The charts rendered in the Behavioral Analysis tab are conditional on which columns were detected—different uploaded datasets produce different visualization sets.

3.3 Predictive Modeling

When a `dropout_flag` column is detected, the dashboard automatically:

1. Selects all numeric features (excluding identifier and timestamp columns)
2. Splits data 70/30 with stratification
3. Standardizes features
4. Trains a logistic regression with class-weight balancing
5. Reports accuracy, precision, recall, confusion matrix, and ranked feature coefficients

This provides a baseline dropout model that non-technical users can obtain simply by uploading their data.

3.4 Stress Testing Protocol

To validate the “any data works” claim, we generated five structurally diverse synthetic datasets (Table 2) and verified that the dashboard successfully loads, maps columns, and renders appropriate charts for each.

Table 2: Stress-test dataset configurations.

Dataset	Logs	Users	Distinguishing Feature
stress_1_default	2000	130	Baseline configuration
stress_2_small_sample	30	30	Small-N edge case
stress_3_high_difficulty	1500	100	Elevated difficulty across all stages
stress_4_low_familiarity	2000	130	Users skewed low-familiarity

Dataset	Logs	Users	Distinguishing Feature
stress_5_ex- tra_wide	3000	200	High-volume stress test

4 4. Results

4.1 4.1 Simulation Output

The simulation engine produces approximately 2000 behavioral logs per run with consistent distributions across stages. Across the default configuration, the overall dropout rate is ~41%, with dropout probability increasing monotonically with stage number—consistent with the cognitive-load hypothesis that higher complexity drives higher abandonment.

4.2 4.2 Dashboard Demonstration

The schema-adaptive dashboard successfully processes all five stress-test datasets without manual configuration. Column detection accuracy is 100% for datasets using the standard schema; the fuzzy matcher handles common aliases (e.g., mapping `Duration_sec` → `dwelling_time`).

4.3 4.3 Predictive Model Performance

The logistic regression baseline achieves approximately 70–80% accuracy on the dropout prediction task (varies by dataset), with information density and stage difficulty consistently ranking as the most important features—further supporting the complexity–dropout relationship. Feature importance interpretation follows common practices in interpretable machine learning, where model coefficients provide transparent explanations of predictive factors (Agarwal et al. 2019).

5 5. Limitations and Future Work

5.1 Current Limitations

1. **Synthetic data only.** All validation uses simulated behavioral logs. The simulation parameters, while grounded in cognitive-load theory, have not been calibrated against human behavioral data.
2. **Binary dropout.** Real-world attention decay is continuous; survival analysis would better model time-to-dropout.

3. **Single-model baseline.** The predictive modeling tab only trains logistic regression. Ensemble methods (Random Forest, XGBoost) would provide stronger baselines.
4. **No user evaluation.** The dashboard has not undergone usability testing with target users (journalists, communication researchers).

5.2 Future Work: Empirical Roadmap

5.2.1 Short-Term (Next Semester)

- **IRB-approved human subject study:** 20–30 participants perform information-browsing tasks under controlled complexity conditions. Compare simulated dropout patterns against empirical distributions to calibrate the simulation engine.
- **Expert heuristic evaluation:** 3–5 HCI researchers review the dashboard via think-aloud protocol + System Usability Scale (SUS) (Brooke 1996).

5.2.2 Medium-Term (CHI 2027 Target)

- **Real-world deployment case study:** Partner with one newsroom or university course to collect authentic engagement telemetry via the dashboard.
- **Survival analysis integration:** Replace binary dropout modeling with Cox proportional hazards (Cox 1972) and Kaplan-Meier estimators (Kaplan and Meier 1958) for continuous attention-decay modeling.
- **Multimodal expansion:** Incorporate eye-tracking and click heatmap data streams.

5.2.3 Long-Term Vision

- **Cross-domain transfer:** Apply the same schema-adaptive pipeline to education (learning dropout), e-commerce (cart abandonment), and health (engagement retention).
- **Open-source release:** Package the dashboard as a `pip`-installable toolkit for communication researchers and UX analysts.

6 6. Team Collaboration

Role	Member	Contributions
Team Lead / Architect	Yicheng Jiang	Simulation engine, schema inference architecture, config externalization, integration, stress testing

Role	Member	Contributions
Data Analyst / Modeler	Shuoyang Jin	SQL queries, adaptive query generation, logistic regression pipeline, feature importance analysis
Frontend / Dashboard	Jiaxin Wang	Streamlit 4-tab dashboard, schema-adaptive chart rendering, CSV upload & column mapping UI, deployment
Narrative / Communication	Quanquan Lu	Theoretical framing (Information Foraging + Cognitive Load), innovation statement, FINAL_REPORT, literature review, AI disclosure

The team followed a Git branching workflow: each member developed independently on `feature/<name>` branches, opening Pull Requests for review by the team lead before merging to `main`.

7 7. Development Challenges and Solutions

During the development of InfoFlow Simulator, we encountered several technical and workflow-related bottlenecks while transforming the initial prototype into a complete telemetry analytics system. These challenges emerged from different stages of development, including data processing, dashboard generalization, environment configuration, version control, and interdisciplinary collaboration.

Rather than being isolated technical issues, these problems were interconnected. Solving them required continuous debugging, communication between team members, and iterative improvements to both the system architecture and our development workflow.

7.1 7.1 Data Processing and Analytical Pipeline Challenges

One major challenge was transforming raw behavioral telemetry logs into reliable analytical data.

The simulation engine generated detailed interaction-level records, including user actions, timestamps, dwell time, and dropout events. However, these raw logs were not immediately suitable for statistical analysis and predictive modeling because different analytical tasks required different levels of data granularity.

During the SQL analysis stage, we encountered an important statistical inconsistency. Initial dropout rate calculations showed a 0% clearance rate for several stages. After debugging, we discovered that the issue was caused by incorrectly grouping exit records using the current

`level_id` rather than tracking the starting stage of each user session. This resulted in incorrect aggregation logic and misleading analytical results.

Another challenge was the mismatch between telemetry records and prediction targets. The raw behavioral logs were recorded at the single-action level, while `dropout_flag` represented a session-level outcome. Directly using action-level records for prediction introduced the risk of inaccurate feature construction and potential information leakage. Therefore, we needed to redesign the feature engineering process by aggregating behavioral records at the session level and ensuring that only information available before dropout prediction was included.

In addition, we encountered technical issues during data storage and processing. When importing CSV files into SQLite, automatic data type inference sometimes caused unexpected aggregation problems. We also needed to handle issues related to data cleaning, encoding formats, and compatibility between Python libraries.

Through repeated validation and debugging, we established a more reliable pipeline from telemetry generation to SQL analysis and predictive modeling. Shuoyang Jin contributed significantly to solving these issues through database analysis, feature construction, and model development.

7.2 7.2 Dashboard Generalization and Data Compatibility Challenges

Another major challenge was improving the dashboard from a fixed-schema prototype into a more flexible analytical system.

The initial implementation depended on predefined column names. However, real-world behavioral datasets often contain inconsistent naming conventions, abbreviations, synonyms, or different semantic expressions. For example, variables representing the same concept may appear as `dwelling_time`, `duration`, or `reading_time`.

This created a limitation: although different datasets might contain equivalent behavioral information, the dashboard could not automatically recognize them without manual modification.

To address this issue, we developed a schema-adaptive mechanism that automatically identifies column meanings and maps different input formats into standardized semantic categories. Implementing this mechanism required solving several engineering problems.

First, we encountered file path and environment compatibility issues. Different operating systems and terminal environments produced errors such as `FileNotFoundError` and Bash path parsing problems when loading datasets.

Second, we faced dependency and module recognition problems. Some functions used in the dashboard were initially unclear regarding whether they belonged to internal modules or external libraries, causing import errors during testing. We resolved these problems by reviewing dependencies, restructuring imports, and improving environment configuration.

Third, when testing datasets containing non-English column names, we needed to ensure correct UTF-8 encoding handling to prevent data corruption during file reading and semantic matching.

Finally, we encountered Python environment isolation problems where project modules could not be recognized because the project root directory was not correctly included in the execution path.

Through continuous testing and debugging, we improved the robustness of the dashboard and successfully transformed it from a dataset-specific visualization tool into a schema-adaptive analytics framework. Jiaxin Wang contributed to solving these challenges through dashboard implementation, visualization development, and data compatibility testing.

7.3 7.3 System Integration and Development Workflow Challenges

Beyond individual technical modules, integrating different components into a unified system was another important challenge.

Because different team members developed separate components independently, maintaining consistency between local environments, branches, and the shared repository became necessary.

During development, we encountered several Git-related problems, including SSH authentication failures, HTTPS credential issues, branch synchronization problems, and differences between local repositories, forked repositories, and the upstream repository.

These problems affected code synchronization and delayed integration because team members sometimes worked with different versions of files. Resolving these issues required understanding Git workflows more deeply, including branch management, pull requests, merging strategies, and remote repository configuration.

Jiang Yicheng helped coordinate the integration process, debug SSH and HTTPS connection problems, and assist team members when repository synchronization issues occurred. Through communication and shared troubleshooting, we gradually established a more stable workflow based on feature branches, pull requests, and regular synchronization with the main branch.

This process also improved our understanding that successful software development depends not only on writing functional code but also on maintaining reliable collaboration infrastructure.

7.4 7.4 Learning New Technical Workflows Through Collaboration

As an interdisciplinary project combining communication research, programming, and data analysis, another challenge was adapting to unfamiliar technical workflows.

Team members entered the project with different levels of programming experience. Therefore, learning programming environments, debugging methods, Git operations, and software development practices required continuous adaptation.

For Quanquan Lu, who had limited programming experience before this project, many parts of the development workflow were initially unfamiliar. During the early stage, he encountered difficulties understanding project structures, running code, and participating in collaborative development.

One practical challenge was learning proper Git collaboration practices. At the beginning, development work was directly conducted on the main branch because the branching workflow was not yet fully understood. Later, after communicating with other team members and learning from their previous experience, the workflow was adjusted to feature branches and pull requests, which improved development safety and code management.

Another challenge was unstable network conditions during synchronization, which occasionally prevented successful data and code updates between local environments and remote repositories. Through discussion with teammates, we identified possible causes and adopted alternative synchronization approaches.

With continuous communication, guidance from teammates, and assistance from AI tools for debugging and technical explanations, Quanquan gradually became familiar with the complete development workflow and was able to contribute effectively to documentation, theoretical framing, and report development.

This experience demonstrated the importance of knowledge sharing in interdisciplinary teams. Team members not only divided tasks but also helped each other overcome technical barriers throughout the development process.

7.5 7.5 Lessons Learned

The development process of InfoFlow Simulator showed that building a computational research prototype requires continuous problem solving beyond individual programming tasks.

The challenges we encountered were closely connected: incorrect data aggregation could affect analytical validity, limited schema flexibility could reduce system usability, and unstable collaboration workflows could slow down integration.

By repeatedly debugging problems, discussing solutions, and improving our workflow, we learned the importance of reproducible development practices, modular system design, and interdisciplinary collaboration.

These experiences shaped both the final system and our understanding of how computational methods can support communication research.

8 8. Conclusion

InfoFlow Simulator demonstrates a **schema-adaptive approach to behavioral telemetry analytics**: any CSV with user-interaction logs can be uploaded, auto-analyzed, and modeled without manual configuration. This lowers the technical barrier for communication researchers and newsroom analysts who need data-driven insights into audience attention and dropout patterns.

The current prototype is a methodological foundation. With the addition of human-subject validation, survival analysis, and a real-world deployment case study, this pipeline forms the core of a competitive CHI 2027 submission.

9 9. AI Usage Disclosure

AI coding assistants (ChatGPT, GitHub Copilot) were used during this project for boilerplate code generation, debugging, and library syntax explanation. All AI-generated code was reviewed, tested, and understood by team members before integration. The final report narrative and theoretical framing were written by the team. See `AI_DISCLOSURE.md` for the complete statement.

References

- Agarwal, S et al. 2019. “Interpretability of Machine Learning Models for Dropout Prediction.” *Proceedings of the ACM Conference on Learning at Scale*.
- Brooke, John. 1996. “SUS: A Quick and Dirty Usability Scale.” *Usability Evaluation in Industry*, 189–94.
- Cox, David R. 1972. “Regression Models and Life-Tables.” *Journal of the Royal Statistical Society: Series B (Methodological)* 34 (2): 187–202.
- Epstein, Ziv, Hause Lin, Gordon Pennycook, and David Rand. 2022. “Quantifying Attention via Dwell Time and Engagement in a Social Media Browsing Environment.” *arXiv*.
- Few, Stephen. 2006. “Dashboard Design for Real-Time Situation Awareness.” *Intelligent Enterprise*.
- Kaplan, Edward L, and Paul Meier. 1958. “Nonparametric Estimation from Incomplete Observations.” *Journal of the American Statistical Association* 53 (282): 457–81.

- Khot, R et al. 2018. “Information Complexity and User Attention: A Lab Study of News Reading Behavior.” *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*.
- Liu, Chao, Ryen W White, and Susan Dumais. 2014. “Understanding Web Browsing Behaviors Through Weibull Analysis of Dwell Time.” *Proceedings of the 37th International ACM SIGIR Conference*.
- Paas, Fred, Alexander Renkl, and John Sweller. 2003. “Cognitive Load Theory and Instructional Design: Recent Developments.” *Educational Psychologist* 38 (1): 1–4.
- Pirolli, Peter. 2007. *Information Foraging Theory: Adaptive Interaction with Information*. Oxford University Press.
- Pirolli, Peter, and Stuart Card. 1999. “Information Foraging.” *Psychological Review* 106 (4): 643–75.
- Saraiya, Purvi, Chris North, and Karen Duca. 2005. “An Insight-Based Methodology for Evaluating Bioinformatics Visualizations.” *IEEE Symposium on Information Visualization*.
- Shneiderman, Ben. 1996. “The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations.” *IEEE Symposium on Visual Languages*, 336–43.
- Sweller, John. 1988. “Cognitive Load During Problem Solving: Effects on Learning.” *Cognitive Science* 12 (2): 257–85.
- Yi, Xing, Liangjie Hong, Erheng Zhong, Nanthan Nan Liu, and Suju Rajan. 2014. “Dwell Time as a Measure of User Interest and Engagement.” *Proceedings of the 23rd International Conference on World Wide Web*.